

GeCode CSP Pipeline - Introduccion

Sistema de Procesamiento de Problemas de Constraint Programming

Proyecto GeCode CSP Pipeline

2026

Contents

Introduccion	3
?Que es GeCode CSP Pipeline?	3
Proposito del Proyecto	3
Caracteristicas Principales	3
? Validacion y Analisis	3
? Propagacion de Restricciones	3
? Resolucion CSP	3
? Persistencia y Utilidades	3
? Compilacion Flexible	3
Tipos de Datos Soportados	4
Operadores Soportados	4
Aritmeticos	4
Logicos	4
Relacionales	4
Conjuntos	4
Funciones Matematicas	4
Compilacion e Instalacion	6
Requisitos del Sistema	6
Para Herramientas del Pipeline	6
Para Integracion con Gecode	6
Instalacion de Dependencias	6
Ubuntu/Debian	6
Fedora/RHEL	6
Instalacion de Gecode Estatico	6
Compilacion del Proyecto	7
Compilacion Completa	7
Compilacion Selectiva	7
Compilacion Monolitica	7
Verificacion de la Instalacion	8
Inicio Rapido	9
Ejemplo Basico	9

1. Crear un archivo JSON de entrada	9
2. Ejecutar el pipeline	9
3. Revisar resultados	9
Pipeline con Persistencia	9
Uso Individual de Componentes	10
Explorar Ejemplos	10
Proximos Pasos	11

Introduccion

?Que es GeCode CSP Pipeline?

GeCode CSP Pipeline es un sistema completo de procesamiento para problemas de **Constraint Satisfaction Problems (CSP)** que integra herramientas de validacion, analisis y resolucion mediante Gecode.

El pipeline procesa problemas definidos en formato JSON, pasando por multiples etapas de verificacion y optimizacion, hasta obtener soluciones completas usando el solver Gecode.

Proposito del Proyecto

Este sistema fue disenado para:

1. **Validar** especificaciones de problemas CSP en formato JSON
2. **Analizar** y transformar expresiones en grafos AST
3. **Verificar** la consistencia de restricciones
4. **Propagar** restricciones usando algoritmos como AC-3
5. **Resolver** problemas CSP completos con Gecode
6. **Persistir** resultados intermedios opcionalmente en SQLite

Caracteristicas Principales

? Validacion y Analisis

- **SyntaxChecker**: Validacion robusta de sintaxis JSON
- **JsonToGraph**: Construccin de AST y grafos de restricciones
- **FunctionChecker**: Verificacion de funciones definidas por el usuario

? Propagacion de Restricciones

- **FwdConsistency**: Implementacion del algoritmo AC-3 (Arc Consistency)
- **BwdConsistency**: Analisis de intervalos y proyeccion inversa
- **ForwardChain**: Encadenamiento hacia adelante para inferencia logica

? Resolucion CSP

- **TestGecodeBridge**: Puente a Gecode para resolucion completa
- **Soporte de 4 tipos**: integer, float, logic, set
- **Multiples soluciones**: Búsqueda exhaustiva o primera solucion

? Persistencia y Utilidades

- **JsonSink**: Almacenamiento en SQLite
- **JsonSource**: Recuperacion desde SQLite
- **CSPEval**: Evaluador de expresiones

? Compilacion Flexible

- **Pipeline tools**: Compilacion sin Gecode
- **Monolitica**: Ejecutables standalone de ~8-12 MB

- **Estatica/Dinamica:** Soporte para linkeo estatico o dinamico de Gecode

Tipos de Datos Soportados

Tipo	Descripcion	Formato JSON
integer	Numeros enteros	{"domain": [1, 100], "value": 10}
float	Numeros de punto flotante	{"domain": [0.0, 50.0], "value": 15.5}
logic	Valores booleanos	{"domain": [true, false], "value": true}
set	Conjuntos de valores	{"domain": {"miembros": ["A", "B"]}, "value": "A"}

Operadores Soportados

Aritmeticos

- + Suma
- - Resta
- * Multiplicacion
- / Division

Logicos

- AND Conjuncion logica
- OR Disyuncion logica
- NOT Negacion
- IMPLICA Implicacion

Relacionales

- = Igualdad
- <> Desigualdad
- < Menor que
- > Mayor que
- <= Menor o igual
- >= Mayor o igual

Conjuntos

- UNION Union de conjuntos
- INTERSECT Interseccion
- DIFFERENCE Diferencia
- SUBSET Subconjunto
- CARDINALITY Numero de elementos
- IN Pertenencia

Funciones Matematicas

El sistema incluye la biblioteca **MiniMath** con funciones:

- `abs(x)` - Valor absoluto
- `sqrt(x)` - Raiz cuadrada
- `sqr(x)` - Cuadrado
- `sin(x)`, `cos(x)`, `tan(x)` - Funciones trigonometricas
- `ln(x)` - Logaritmo natural
- `exp(x)` - Exponencial
- `pow(x, y)` - Potencia

Compilacion e Instalacion

Requisitos del Sistema

Para Herramientas del Pipeline

- **Free Pascal Compiler** (fpc) ≥ 3.0
- **gcc** ≥ 7.0
- **make**
- **SQLite3** (libsqlite3-dev)

Para Integracion con Gecode

- **g++** ≥ 7.0
- **Gecode** 6.x (estatico o dinamico)
- **Free Pascal Compiler** (fpc) ≥ 3.0

Instalacion de Dependencias

Ubuntu/Debian

```
# Herramientas basicas
sudo apt install build-essential fpc gcc make libsqlite3-dev

# Para Gecode (version dinamica del sistema)
sudo apt install libgecode-dev

# Para compilar PDFs (opcional)
sudo apt install pandoc texlive-latex-base texlive-fonts-recommended texlive-latex-extra
```

Fedora/RHEL

```
# Herramientas basicas
sudo dnf install gcc fpc make sqlite-devel

# Para compilar PDFs (opcional)
sudo dnf install pandoc texlive-scheme-basic
```

Instalacion de Gecode Estatico

Para crear ejecutables verdaderamente standalone, se recomienda compilar Gecode estatico:

```
cd /tmp
wget https://github.com/Gecode/gecode/archive/refs/tags/release-6.3.0.tar.gz
tar xzf release-6.3.0.tar.gz
cd gecode-release-6.3.0

./configure \
  --prefix=$HOME/gecode-static \
  --disable-shared \
```

```

--enable-static \
--disable-examples \
--disable-qt \
--disable-gist

make -j$(nproc)
make install

# Configurar variable de entorno
export GECODE_HOME=$HOME/gecode-static
echo "export GECODE_HOME=$HOME/gecode-static" >> ~/.bashrc

```

Compilacion del Proyecto

Compilacion Completa

```

# Clonar o descargar el proyecto
cd gecode

# Compilar todo (pipeline + gecode)
make

# Verificar ejecutables generados
ls -lh bin/

```

Compilacion Selectiva

```

# Solo herramientas del pipeline (sin Gecode)
make pipeline

# Solo herramientas Gecode
make gecode

# Limpieza
make clean      # Solo objetos
make distclean  # Todo

```

Compilacion Monolitica

Para crear ejecutables totalmente standalone:

```
./scripts/build_monolithic.sh src/TestGecodeBridge.pas
```

Esto genera un ejecutable que incluye:

- ? Todo el codigo Pascal compilado
- ? Puente C++ a Gecode
- ? Gecode linkeado estaticamente
- ? Sin dependencias de libgecode*.so

- ? Tamano: ~8-12 MB

Verificacion de la Instalacion

```
# Probar herramientas del pipeline
./bin/SyntaxChecker ejemplos/Json_input_1.json
./bin/JsonToGraph ejemplos/Json_input_1.json
./bin/FwdConsistency graph.json

# Probar integracion Gecode
./bin/TestGecodeBridge graph.json

# Ejecutar pipeline completo
./scripts/pipeline.sh ejemplos/Json_input_1.json
```

Si todos los comandos ejecutan sin errores, la instalacion fue exitosa.

Inicio Rapido

Ejemplo Basico

1. Crear un archivo JSON de entrada

Crear `mi_problema.json`:

```
{
  "precision": 2,
  "variables": [
    {
      "nombre": "x",
      "tipo": "integer",
      "domain": [1, 10],
      "value": null
    },
    {
      "nombre": "y",
      "tipo": "integer",
      "domain": [1, 10],
      "value": null
    }
  ],
  "expresiones": [
    "x + y = 10",
    "x > y"
  ]
}
```

2. Ejecutar el pipeline

```
./scripts/pipeline.sh mi_problema.json
```

3. Revisar resultados

El pipeline ejecutara todas las etapas y mostrara:

- ? Validacion de sintaxis
- ? Construccion de AST
- ? Verificacion de funciones
- ? Propagacion de restricciones
- ? Soluciones encontradas por Gecode

Pipeline con Persistencia

```
# Persistir cada etapa en SQLite
./scripts/pipeline.sh --db resultados.db --tag prueba_1 mi_problema.json

# Procesar multiples archivos
```

```
./scripts/pipeline.sh --db resultados.db ejemplos/*.json
```

```
# Recuperar resultados de una etapa
```

```
./bin/JsonSource resultados.db prueba_1_syntax
```

```
./bin/JsonSource resultados.db prueba_1_graph
```

```
./bin/JsonSource resultados.db prueba_1_csp
```

Uso Individual de Componentes

```
# 1. Validar sintaxis
```

```
./bin/SyntaxChecker ejemplos/Json_input_1.json
```

```
# 2. Construir grafo AST
```

```
./bin/JsonToGraph ejemplos/Json_input_1.json > graph.json
```

```
# 3. Verificar funciones
```

```
./bin/FunctionChecker graph.json > graph_checked.json
```

```
# 4. Propagacion forward
```

```
./bin/FwdConsistency graph_checked.json > graph_fwd.json
```

```
# 5. Propagacion backward
```

```
./bin/BwdConsistency graph_fwd.json > graph_bwd.json
```

```
# 6. Resolucion con Gecode
```

```
./bin/TestGecodeBridge graph_bwd.json > solucion.json
```

Explorar Ejemplos

El directorio ejemplos/ contiene multiples archivos de prueba:

```
# Listar ejemplos disponibles
```

```
ls -l ejemplos/Json_input_*.json
```

```
# Ver contenido de un ejemplo
```

```
cat ejemplos/Json_input_1.json
```

```
# Ejecutar un ejemplo
```

```
./scripts/pipeline.sh ejemplos/Json_input_2.json
```

```
# Ejecutar todos los ejemplos
```

```
for f in ejemplos/Json_input_*.json; do
```

```
    echo "=== Procesando $f ==="
```

```
    ./scripts/pipeline.sh "$f"
```

```
    echo
```

```
done
```

Proximos Pasos

1. Leer **02_arquitectura.md** para entender el diseno del sistema
2. Leer **03_componentes_pipeline.md** para detalles de cada componente
3. Leer **04_integracion_gecode.md** para entender el puente a Gecode
4. Revisar los reportes de pruebas en **docs/reporte_*.md**

Nota: Esta es la version inicial del manual. Para informacion detallada sobre cada componente, consultar los documentos especificos en **docs/**.